Soporte técnico de una red de oficinas

Sistema multiagente con reglas de seguridad en código · Alberto Olmos · Práctica de Laboratorio I, IA Agéntica · octubre 2026

1. Problema y alcance

El sistema hace de **soporte de primer nivel** para los empleados de cuatro oficinas (MAD-01, MAD-02, BCN-01, VAL-01). Entra el texto de la incidencia, la oficina y un ticket opcional; sale un **Resultado** con el estado final (*resuelto*, *escalado*, *rechazado* o *tope_alcanzado*), la respuesta al empleado con citas a la base de conocimiento y la traza completa. Lo que no se resuelve se escala a segundo nivel con un resumen de lo probado. Queda fuera lo que no es soporte de oficinas (se rechaza) y cualquier reparación física. Está escrito en Python 3.12 **sin frameworks de agentes**: el bucle y las reglas son código propio que puedo explicar y probar.

2. Arquitectura multiagente

Agente	Decide	Salida (JSON validado)	Herramientas
clasificador	tipo y prioridad	tipo, prioridad, motivo	buscar_tickets
diagnóstico	causa probable	causa, servicio, evidencias	consultar_servicio, consultar_equipo, buscar_tickets
conocimiento	procedimiento	procedimiento, citas, accion	buscar_docs; *propone* reiniciar_servicio y cerrar_ticket
escalado	traspaso a nivel 2	resumen, probado, prioridad	crear_escalado

Orquestación por traspaso: clasificador → diagnóstico → conocimiento → resuelto o escalado. El soporte es secuencial (no se puede proponer una solución sin conocer la causa), así que un supervisor solo añadiría llamadas al modelo y puntos de fallo. Las excepciones son reglas fijas en código: ticket reabierto o avería en varias oficinas → escalado tras el diagnóstico; sin cita válida o con la acción rechazada → escalado.

Estado y mensajes: un `Estado` compartido guarda lo averiguado (tipo, prioridad, causa, servicio, citas, acciones, traza y pasos). Cada agente recibe un resumen JSON de ese estado, no la conversación entera: menos texto, menos créditos y menos confusión. En cada turno el modelo devuelve un único JSON (llamada a herramienta o salida final); el código valida el contrato y su prompt se genera desde el registro, con **solo** las herramientas permitidas a ese agente (mínimo privilegio).

Prioridades. Las propone el clasificador; el código la **sube** si los datos exigen más (nunca la baja) y un rechazo no tiene prioridad. Servicios críticos: vpn, correo, erp, ficheros e intranet (la intranet aloja el autoservicio de contraseñas: si falla es un problema de seguridad). No críticos: impresión y wifi de invitados.

	Criterio	Ejemplo en los datos	1.ª respuesta
P1	Afecta a más de una oficina, o un servicio crítico está caído	vpn caída en MAD-01; erp caído en MAD-02 y BCN-01	15 min
P2	Afecta a una oficina entera, o un crítico está degradado	correo degradado en BCN-01	30 min
P3	Un empleado no puede trabajar y no tiene alternativa	PC sin red en VAL-01 (tarjeta averiada)	2 h
P4	Molestia con alternativa	una impresora (hay otras), el equipo va lento	1 día laborable

3. Modelos de lenguaje

Los cuatro modelos cumplen el mismo contrato, `generar(sistema, mensajes) → texto`, así que el resto del sistema no sabe cuál usa y las reglas se aplican igual con todos.

Modelo	Qué es	Para qué se usa	Qué necesita
ScriptedModel	Respuestas fijadas de antemano	Las 289 pruebas y la evaluación	Nada
ModeloSimulado	Reglas por palabras clave, determinista	Demo sin token y web (por defecto)	Nada
ModeloOllama (qwen2.5:14b)	Modelo real ejecutado en mi ordenador con Ollama	Las dos trazas reales del informe	Gratis: instalar Ollama y el modelo (9 GB)
ModeloHF (Qwen2.5-72B)	Modelo real en los servidores de Hugging Face	Opcional	Token y créditos de pago

El plan inicial era Hugging Face, pero ya no da créditos gratuitos: dos cuentas devolvieron *402 Payment Required*. Como indicó el profesor en clase, usé Ollama, que ejecuta el modelo en local sin token ni coste. El código de Hugging Face sigue disponible con `--modelo hf`. Para corregir no hace falta ninguno de los dos: las pruebas y el README usan el simulado.

4. Herramientas y conocimiento

Siete herramientas en un registro con **descripción** y marca de **reversible**; todas pasan por un único punto (`ejecutar`) que convierte cualquier fallo en `ErrorHerramienta`. Las dos irreversibles (reiniciar_servicio, cerrar_ticket) nunca se ejecutan dentro del bucle de un agente: conocimiento solo las propone. Los datos de la red se cargan en memoria, así que cada ejecución parte del mismo estado.

La recuperación es un **BM25 escrito a mano** sobre 15 documentos propios (11 artículos kb-* con Síntomas, Causa y Pasos, y 4 políticas: catálogo de servicios, escalado, reinicios y prioridades). Cada fragmento es una sección, así que las citas son precisas (`docs/kb-red-01.md#Pasos`). Lo elegí frente a embeddings porque no necesita red ni dependencias, es determinista y explicable. Limitación: no entiende sinónimos; se mitiga con frases de empleados en cada sección de síntomas.

5. Robustez y seguridad

Regla (en código)	Qué garantiza
R1 Confirmación	Una acción irreversible solo se ejecuta si confirmar() devuelve verdadero. Si la persona dice que no, no se ejecuta y se escala.
R2 Guardarraíl	Un servicio crítico en horario laboral solo se reinicia si es el diagnosticado y está caído o degradado; nunca en otra oficina; máximo un reinicio por servicio. Si bloquea, ni siquiera se pregunta.
R3 Tope	12 pasos (respuestas del modelo y herramientas): evita bucles y limita el gasto. Al superarlo, tope_alcanzado.
R4 Errores	Errores de herramienta, herramienta no permitida y JSON ilegible son observaciones; dos ilegibles seguidas → escalado.
R5 Citas	Una cita solo vale si ese fragmento lo devolvió buscar_docs en esta ejecución; las inventadas se borran. Sin cita válida no hay resuelto.

Además: la prioridad la sube el código si los datos lo exigen (un crítico caído es P1 aunque el modelo diga P3) y nunca la baja; la respuesta al empleado la redacta una plantilla, así que el modelo no puede prometer plazos. En Claude Code, `settings.json` permite solo leer y probar, pide permiso para dependencias, ejecuciones que gastan créditos y commits, y prohíbe leer `.env` (también por terminal), borrar, publicar y sacar datos. El token vive solo en mi `.env`: no está en el código ni en la entrega.

6. Pruebas y evaluación

289 pruebas con **dobles del modelo** (ScriptedModel con respuestas fijadas y un cliente falso para Hugging Face): deterministas, sin red y sin token. Cada regla R1–R5 tiene pruebas que fallan si se rompe; se comprobó rompiendo cada regla a propósito (mutaciones).

La **evaluación** son 11 casos cuyo resultado esperado definí yo aplicando mis reglas, sin mirar el código (oráculo independiente). Mide el **resultado** (final, tipo, prioridad) y la **trayectoria** (C = clasificador, D = diagnóstico, K = conocimiento, E = escalado). Se ejecuta con guion (comprueba las reglas) y con el modelo simulado (lo que vería un usuario sin token).

#	Caso	Esperado: final · tipo · prioridad · trayectoria
1	«no tengo internet», MAD-01, confirma (vpn caída)	resuelto · red · P1 · C-D-K
2	Igual, pero rechaza el reinicio (R1)	escalado · red · P1 · C-D-K-E
3	«no tengo internet», VAL-01 (tarjeta rota): mismo síntoma, otra causa	resuelto · red · P3 · C-D-K
4	Impresora, MAD-02, T-0007 reabierto	escalado · impresora · P4 · C-D-E
5	ERP caído en MAD-02 y BCN-01; el modelo dice P2	escalado · acceso_aplicacion · P1 · C-D-E
6	Correo degradado, BCN-01; el modelo dice P4	resuelto · correo · P2 · C-D-K
7	«¿me recomiendas un restaurante?»	rechazado · fuera_de_ambito · — · C
8	Oficina inexistente (SEV-01)	rechazado · — · — · sin agentes
9	Cita inventada por el modelo (R5)	escalado · red · P1 · C-D-K-E
10	Propone reiniciar el ERP con la vpn diagnosticada (R2)	escalado · red · P1 · C-D-K-E
11	«me va lento el ordenador» (caso propio)	resuelto · hardware · P4 · C-D-K

Primera pasada: 9/11 con guion y 6/9 con el simulado. No cambié lo esperado; analicé cada fallo. Uno era del código (un rechazo conservaba la prioridad P4 del clasificador: lo que no se tramita no debe tener prioridad), otro de contenido (no había artículo de equipos lentos) y tres del simulado (no reconocía lo que está fuera de ámbito ni «ordenador», y empezaba siempre en P3). El caso 11 con el simulado **acertaba el final por casualidad**: «resuelto» citando el artículo de la tarjeta de red. Por eso se evalúa también la trayectoria y los datos. Tras los arreglos: **11/11 y 9/9**.

7. Dos trazas reales

Con Ollama (sección 3, `--modelo ollama`). Con **qwen2.5:7b** las primeras trazas acabaron escaladas: el modelo devolvía JSON fuera de contrato (R4), no consultaba herramientas y se inventaba citas (R5 las eliminó). Las reglas evitaron cualquier daño, pero el prompt no decía el formato exacto, los valores permitidos ni qué es cada servicio. Lo añadí (generado desde las mismas listas que valida el código) y pasé a **qwen2.5:14b**. Estas son las trazas finales, tal cual las imprime `--traza`; coinciden con mis casos 1 y 5.

Traza 1. «no tengo internet» en MAD-01: pide confirmación del reinicio de la vpn (respondo «s»).

```
¿Confirmas reiniciar_servicio(servicio=vpn, oficina=MAD-01)? [s/N] Final: resuelto
Tipo: red · Prioridad: P1

Respuesta al empleado:
1. Consultar el estado del servicio `vpn` en la oficina del empleado.
2. Comprobar si otras oficinas tienen también la `vpn` caída; si es así,
   la avería es multioficina: escalar con P1 sin reiniciar nada.
3. Si solo afecta a esta oficina y el servicio está `caido` o `degradado`,
   puede solicitarse un reinicio de `vpn` según `politica-reinicios.md`
   (requiere confirmación; en horario laboral solo si es el servicio diagnosticado).
4. Tras el reinicio, pedir al empleado que cierre y vuelva a abrir el navegador.
5. Si en 10 minutos no se recupera, escalar a segundo nivel.

Fuentes: docs/kb-red-01.md#Causa, docs/kb-red-01.md#Pasos

Traza:
   1. [clasificador] respuesta -> {"tipo": "red", "prioridad": "P2", "motivo": "El empleado no puede
      acceder a internet, lo que afecta su capacidad de trabajo, pero no se especifica si es un ser…
   2. [diagnostico] respuesta -> {"herramienta": "consultar_servicio", "argumentos": {"servicio": "vpn",
      "oficina": "MAD-01"}}
   3. [diagnostico] consultar_servicio({"servicio": "vpn", "oficina": "MAD-01"}) -> {"servicio": "vpn",
      "oficina": "MAD-01", "estado": "caido"}
   4. [diagnostico] respuesta -> {"causa": "El servicio VPN de la oficina MAD-01 está caído, lo que
      impide la conexión a internet.", "servicio": "vpn", "evidencias": ["El servicio VPN está caíd…
   5. [orquestador] nota -> prioridad subida de P2 a P1: servicio vpn afectado en MAD-01
   6. [conocimiento] respuesta -> {"herramienta": "buscar_docs", "argumentos": {"consulta": "El servicio
      VPN de la oficina MAD-01 está caído, lo que impide la conexión a internet.", "k": 3}}
   7. [conocimiento] buscar_docs({"consulta": "El servicio VPN de la oficina MAD-01 está caído, lo que
      impide la conexión a internet.", "k": 3}) -> {"fragmentos": [{"doc": "docs/kb-red-01.md", "seccion":
       "Causa", "texto": "El túnel VPN de la oficina con el centro de datos está caído. El tráfico\
      ncorporativo…
   8. [conocimiento] respuesta -> {"procedimiento": "1. Consultar el estado del servicio `vpn` en la
      oficina del empleado.\n2. Comprobar si otras oficinas tienen también la `vpn` caída; si es as…
```

```
   9. [conocimiento] reiniciar_servicio({"servicio": "vpn", "oficina": "MAD-01"}) -> {"servicio": "vpn",
      "oficina": "MAD-01", "estado_anterior": "caido", "estado": "ok"}
```

Traza 2. «no puedo entrar en el ERP» en MAD-02: avería en varias oficinas, escalado P1.

```
Final: escalado
Tipo: acceso_aplicacion · Prioridad: P1
Escalado: ESC-0001

Respuesta al empleado:
Tu incidencia se ha escalado a segundo nivel con el identificador ESC-0001 (prioridad P1). Tiempo de
      primera respuesta: 15 minutos.

Traza:
   1. [clasificador] respuesta -> {"tipo": "acceso_aplicacion", "prioridad": "P1", "motivo": "El usuario
      no puede acceder al ERP, un servicio crítico."}
   2. [diagnostico] respuesta -> {"herramienta": "consultar_servicio", "argumentos": {"servicio": "erp",
      "oficina": "MAD-02"}}
   3. [diagnostico] consultar_servicio({"servicio": "erp", "oficina": "MAD-02"}) -> {"servicio": "erp",
      "oficina": "MAD-02", "estado": "caido"}
   4. [diagnostico] respuesta -> {"causa": "El servicio ERP está caído en la oficina MAD-02.", "servicio":
       "erp", "evidencias": ["El servicio ERP está caído en la oficina MAD-02."]}
   5. [orquestador] nota -> escalado directo: avería multioficina de erp: MAD-02, BCN-01
   6. [escalado] respuesta -> {"resumen": "El servicio ERP está caído en la oficina MAD-02, afectando el
      acceso a la aplicación para los usuarios locales.", "probado": ["reiniciar navegador"…
   7. [escalado] crear_escalado({"ticket": null, "resumen": "El servicio ERP está caído en la oficina MAD-
      02, afectando el acceso a la aplicación para los usuarios locales.", "prioridad": "P1"}) -> {
      "escalado": "ESC-0001", "ticket": null, "prioridad": "P1"}
```

8. Forma de trabajo y limitaciones

Dirigí a Claude Code con un **CLAUDE.md** de menos de 200 líneas (contratos, reglas y valores cerrados), permisos de mínimo privilegio y **una orden por hito** con criterios comprobables (hitos 0 a 7b). Las decisiones son mías; Claude Code redactó a partir de ellas y generó el código, que revisé. El `worklog.md` recoge decisiones, problemas, comprobaciones y quién redactó cada cosa.

- El simulado funciona por palabras clave: una frase sin ninguna conocida puede clasificarse mal. Con el modelo real decide el clasificador.
- R5 comprueba que la cita se recuperó, no que sea relevante.
- BM25 no entiende sinónimos; con más documentos convendría una búsqueda híbrida.
- Con un modelo real pequeño, el agente de escalado puede listar en «probado» cosas que nadie probó; no es peligroso, pero sí inexacto.
- El tope de 12 pasos bastó con el modelo real (9 pasos en la traza más larga).

Reproducir: `uv sync` · `uv run pytest` · órdenes del README (modelo simulado por defecto, sin token).